

UniApp【萌宠案例】基于uni-ui扩展组件的实现方案

uni-ui作为UniApp官方推荐的扩展组件库，提供了丰富的跨端组件，兼具美观与实用性。本方案在萌宠案例中引入uni-ui核心组件，覆盖搜索、分类、列表、弹窗等高频场景，通过组件化开发提升页面质感与开发效率，同时确保多端表现一致。方案将先完成uni-ui环境配置，再分模块实现组件集成与业务逻辑关联。

一、前置准备：uni-ui环境配置

uni-ui组件需通过HBuilderX插件市场安装或npm引入，本方案采用插件市场安装方式，配置步骤如下：

1. 组件安装

1. 打开HBuilderX，在项目管理器中右键点击萌宠案例项目，选择“插件安装” → “安装插件”。
2. 在插件市场搜索“uni-ui”，选择官方“uni-ui组件库”进行安装，安装完成后重启HBuilderX生效。
3. 若需单独引入特定组件（如仅用搜索框和标签），可在“pages.json”中配置easycom规则，实现组件自动按需引入。

2. easycom配置（按需引入）

在项目根目录的“pages.json”中添加easycom配置，无需手动导入组件即可直接使用，减少代码冗余：

Code block

```
1
2  {
3    "easycom": {
4      "autoscan": true,
5      "custom": {
6        // uni-ui组件匹配规则: @/uni_modules/组件名/组件名.vue
7        "^uni-(.*)": "@uni_modules/uni-$1/uni-$1.vue"
8      }
9    },
10   "pages": [
11     {
12       "path": "pages/category/category",
```

```

13     "style": {
14       "navigationBarTitleText": "萌宠分类"
15     }
16   },
17   {
18     "path": "pages/detail/detail",
19     "style": {
20       "navigationBarTitleText": "萌宠详情"
21     }
22   }
23 ]
24 }
25

```



提示：若使用npm引入uni-ui，需先执行“npm install @dcloudio/uni-ui”，再在“main.js”中全局注册或在页面中局部引入，本方案优先推荐插件市场安装以简化配置。

二、核心页面实现：萌宠分类页（uni-ui组件集成）

分类页作为萌宠案例的核心入口，将集成uni-ui的 `uni-search-bar`（搜索框）、`uni-segmented-control`（分类标签）、`uni-list`（列表）、`uni-badge`（徽章）等组件，实现搜索、分类切换、列表展示等功能。

1. 页面结构（pages/category/category.vue）

Code block

```

1
2 <template>
3   <view class="category-page">
4
5     <uni-search-bar
6       placeholder="搜索萌宠名称/品种"
7       v-model="searchValue"
8       @input="handleSearchInput"
9       @confirm="handleSearchConfirm"
10      @cancel="handleSearchCancel"
11      clearabled
12      radius="100rpx"
13      bgColor="#f5f5f5"
14      cancelText="取消"

```

```
15 ></uni-search-bar>
16
17
18 <uni-segmented-control
19   v-model="currentTab"
20   :values="categoryTabs"
21   mode="card"
22   @clickItem="handleTabClick"
23   activeColor="#ff6700"
24   borderColor="#eee"
25   height="80rpx"
26   fontSize="28rpx"
27   class="category-tabs"
28 ></uni-segmented-control>
29
30
31 <uni-load-more
32   v-if="isLoading"
33   status="loading"
34   class="load-more"
35 ></uni-load-more>
36
37
38 <uni-list v-if="petList.length && !isLoading" class="pet-list">
39   <uni-list-item
40     v-for="(pet, index) in petList"
41     :key="pet.id + index"
42     clickable
43     @click="gotoDetail(pet.id)"
44     bordered
45     class="pet-list-item"
46   >
47
48   <template #left>
49     <uni-image
50       :src="pet.coverImg"
51       mode="aspectFill"
52       class="pet-img"
53       placeholder
54       placeholderSrc="/static/placeholder.png"
55     ></uni-image>
56   </template>
57
58   <template #body>
59     <view class="pet-info">
60       <view class="pet-name">{{ pet.name }}</view>
61       <view class="pet-breed">品种: {{ pet.breed }}</view>
```

```

62         <view class="pet-meta">
63
64             <uni-badge
65                 text="热门"
66                 type="warning"
67                 size="mini"
68                 class="hot-badge"
69             ></uni-badge>
70             <view class="like-count">
71                 <uni-icon type="heart" size="24rpx" color="#ff4444"></uni-icon>
72                 {{ pet.likeCount }}
73             </view>
74         </view>
75     </view>
76 </template>
77
78 <template #right>
79     <uni-icon type="right" size="28rpx" color="#999"></uni-icon>
80 </template>
81 </uni-list-item>
82 </uni-list>
83
84
85 <uni-empty
86     v-if="petList.length === 0 && !isLoading"
87     text="暂无匹配的萌宠"
88     mode="list"

```

2. 逻辑实现（pages/category/category.vue）

结合uni-ui组件的事件回调，实现搜索、分类切换、筛选、列表跳转等核心业务逻辑，同时管理组件状态：

Code block

```

1
2 <script setup>
3 import { ref, onLoad } from 'vue'
4 import request from '../utils/request'
5
6 // 搜索相关状态
7 const searchValue = ref('')
8 const isSearching = ref(false)
9
10 // 分类标签状态
11 const currentTab = ref(0)
12 const categoryTabs = ref(['全部', '猫咪', '狗狗', '其他宠物'])
13

```

```
14 // 列表相关状态
15 const petList = ref([])
16 const isLoading = ref(false)
17 const page = ref(1)
18 const pageSize = ref(10)
19
20 // 筛选相关状态
21 const filterPopup = ref(null)
22 const priceTags = ref([
23   { text: '不限', selected: true },
24   { text: '500元以下', selected: false },
25   { text: '500-1000元', selected: false },
26   { text: '1000元以上', selected: false }
27 ])
28 const currentFilter = ref({ price: '不限' })
29
30 // 页面加载：初始化数据
31 onLoad(() => {
32   loadPetData()
33 })
34
35 // 加载萌宠数据（结合分类与筛选条件）
36 const loadPetData = () => {
37   isLoading.value = true
38   // 构建请求参数：分类类型 + 搜索关键词 + 筛选条件 + 分页
39   const params = {
40     type: currentTab.value === 0 ? 'all' :
categoryTabs.value[currentTab.value],
41     keyword: searchValue.value.trim(),
42     priceRange: currentFilter.value.price,
43     page: page.value,
44     pageSize: pageSize.value
45   }
46
47   request.get('/category/pet', params)
48     .then((res) => {
49     if (Array.isArray(res.list)) {
50       // 分页处理：第一页覆盖，后续页追加
51       if (page.value === 1) {
52         petList.value = res.list.filter(item => item.id && item.coverImg)
53       } else {
54         petList.value = [...petList.value, ...res.list.filter(item =>
item.id && item.coverImg)]
55       }
56     }
57   })
58   .catch((err) => {
```

```

59     uni.showToast({ title: err.msg || '数据加载失败', icon: 'error' })
60     // 分页加载失败回滚页码
61     if (page.value > 1) page.value -= 1
62   })
63   .finally(() => {
64     isLoading.value = false
65     isSearching.value = false
66   })
67 }
68
69 // 1. 搜索相关事件
70 // 输入实时搜索（防抖处理）
71 const handleSearchInput = (value) => {
72   searchValue.value = value
73   // 清除之前的定时器
74   if (window.searchTimer) clearTimeout(window.searchTimer)
75   // 300ms防抖
76   window.searchTimer = setTimeout(() => {
77     if (value.trim().length > 0) {
78       page.value = 1
79       loadPetData()
80     }
81   }, 300)
82 }
83
84 // 搜索确认
85 const handleSearchConfirm = () => {
86   if (searchValue.value.trim().length === 0) {

```

3. 样式实现（pages/category/category.vue）

配合uni-ui组件的默认样式进行微调，确保页面布局美观且适配多端：

Code block

```

1
2  .category-page {
3    background-color: #f5f5f5;
4    min-height: 100vh;
5    padding-bottom: 120rpx;
6  }
7
8  /* 搜索框样式 */
9  .uni-search-bar {
10    margin: 20rpx;
11    --uni-search-bar-bg-color: #f5f5f5;
12    --uni-search-bar-radius: 100rpx;

```

```
13     --uni-search-bar-height: 72rpx;
14 }
15
16 /* 分类标签样式 */
17 .category-tabs {
18     margin: 0 20rpx;
19 }
20 .uni-segmented-control {
21     --uni-segmented-control-active-color: #ff6700;
22     --uni-segmented-control-border-color: #eee;
23     --uni-segmented-control-card-border-radius: 100rpx;
24 }
25
26 /* 列表样式 */
27 .pet-list {
28     margin: 20rpx;
29 }
30 .pet-list-item {
31     --uni-list-item-height: auto;
32     --uni-list-item-padding: 20rpx;
33 }
34 .pet-img {
35     width: 140rpx;
36     height: 140rpx;
37     border-radius: 16rpx;
38     margin-right: 20rpx;
39 }
40 .pet-info {
41     flex: 1;
42 }
43 .pet-name {
44     font-size: 28rpx;
45     font-weight: 500;
46     margin-bottom: 8rpx;
47 }
48 .pet-breed {
49     font-size: 24rpx;
50     color: #666;
51     margin-bottom: 12rpx;
52 }
53 .pet-meta {
54     display: flex;
55     align-items: center;
56     gap: 16rpx;
57 }
58 .hot-badge {
59     margin-right: 8rpx;
```

```

60 }
61 .like-count {
62   font-size: 22rpx;
63   color: #999;
64   display: flex;
65   align-items: center;
66   gap: 4rpx;
67 }
68
69 /* 无数据提示 */
70 .empty-tip {
71   margin-top: 200rpx;
72 }
73
74 /* 筛选弹窗样式 */
75 .popup-content {
76   padding: 20rpx;
77   background-color: #fff;
78   border-top-left-radius: 30rpx;
79   border-top-right-radius: 30rpx;
80 }
81 .popup-header {
82   display: flex;
83   justify-content: space-between;
84   align-items: center;
85   margin-bottom: 30rpx;
86 }
87 .header-title {
88   font-size: 22rpx;

```

三、扩展页面：萌宠详情页（uni-ui组件强化）

详情页引入uni-ui的 `uni-gallery`（图片预览）、`uni-rating`（评分）、`uni-goods-nav`（底部导航）等组件，提升详情页的交互体验。

1. 核心结构与逻辑（pages/detail/detail.vue）

Code block

```

1
2 <template>
3   <view class="detail-page">
4
5     <uni-gallery
6       :images="petImages"
7       :current="currentImageIndex"
8       @change="handleGalleryChange"

```

```
9      class="pet-gallery"
10    ></uni-gallery>
11
12
13    <view class="pet-base-info">
14      <view class="pet-title">{{ petInfo.name }}</view>
15      <view class="pet-breed">品种: {{ petInfo.breed }}</view>
16
17      <uni-rating
18        v-model="petInfo.score"
19        disabled
20        size="28rpx"
21        color="#ffd21e"
22        margin="10rpx 0"
23      ></uni-rating>
24      <view class="pet-price">
25        参考价格: <span class="price-num">¥{{ petInfo.price }}</span>
26      </view>
27    </view>
28
29
30    <uni-segmented-control
31      v-model="currentDetailTab"
32      :values="detailTabs"
33      mode="button"
34      @clickItem="handleDetailTabClick"
35      class="detail-tabs"
36    ></uni-segmented-control>
37
38    <view class="detail-content">
39      <view v-if="currentDetailTab === 0" class="content-desc">
40        {{ petInfo.description || '暂无详细介绍' }}
41      </view>
42      <view v-if="currentDetailTab === 1" class="content-params">
43        <uni-list bordered>
44          <uni-list-item title="年龄" :rightText="petInfo.age || '未知'"></uni-
45list-item>
46          <uni-list-item title="性别" :rightText="petInfo.gender || '未知'">
47</uni-list-item>
48          <uni-list-item title="健康状况" :rightText="petInfo.health || '良好'">
49</uni-list-item>
50        </uni-list>
51      </view>
52    </view>
53
54    <uni-goods-nav
```

```

53         :fill="true"
54         :options="goodsNavOptions"
55         :buttonGroup="buttonGroup"
56         @clickOption="handleNavClick"
57         @clickButton="handleBuyClick"
58     ></uni-goods-nav>
59 </view>
60 </template>
61
62 <script setup>
63 import { ref, onLoad } from 'vue'
64 import request from '../utils/request'
65
66 // 图片预览相关
67 const petImages = ref([])
68 const currentIndex = ref(0)
69
70 // 详情标签相关
71 const currentDetailTab = ref(0)
72 const detailTabs = ref(['详细介绍', '基本参数'])
73
74 // 萌宠信息
75 const petInfo = ref({
76     score: 4.5 // 默认评分
77 })
78
79 // 底部导航相关
80 const goodsNavOptions = ref([
81     { icon: 'chat', text: '咨询' },
82     { icon: 'collect', text: '收藏' }
83 ])
84 const buttonGroup = ref([
85     { text: '联系商家', backgroundColor: '#ffb042', color: '#fff' },

```

四、uni-ui组件使用规范与优化技巧

1. 组件使用规范

- **属性传递规范：**组件属性优先使用v-model实现双向绑定（如uni-rating、uni-segmented-control），避免手动监听change事件修改值，简化代码。
- **样式修改规范：**修改uni-ui组件样式时，优先使用组件自带的CSS变量（如--uni-search-bar-bg-color），避免使用!important覆盖默认样式，提升兼容性。
- **事件处理规范：**组件事件回调函数需明确参数含义，如uni-goods-nav的clickButton事件需通过index区分不同按钮，确保逻辑清晰。

2. 性能优化技巧

- **按需引入组件**：通过easycom配置实现组件按需引入，避免全量引入导致包体积增大，尤其在小程序端需控制包体积在2M以内。
- **图片优化**：使用uni-image组件的placeholder和placeholderSrc属性，减少图片加载过程中的空白感，同时开启懒加载提升首屏渲染速度。
- **条件渲染优化**：详情页的标签内容使用v-if而非v-show，避免初始渲染不必要的DOM节点，减少页面加载负担。
- **数据预处理**：将接口返回的图片数组转换为uni-gallery所需格式时，在请求成功后一次性处理，避免在模板中进行复杂数据转换。

五、常见问题与解决方案

问题现象	原因分析	解决方案
组件样式修改不生效	1. 未使用组件自带的CSS变量；2. 样式作用域（scoped）限制；3. 样式优先级低于组件默认样式	1. 优先使用组件CSS变量；2. 如需深度修改，使用::v-deep穿透作用域；3. 避免使用!important，通过层级提升优先级
组件事件无响应	1. 事件名称拼写错误；2. 事件回调函数未正确定义；3. 组件未正确挂载（如ref未关联）	1. 对照uni-ui文档检查事件名称；2. 确保回调函数为Function类型；3. 组件挂载完成后再调用方法（如onReady中操作）
小程序端组件显示异常	1. 组件版本与小程序基础库版本不兼容；2. 包体积过大导致组件加载失败；3. 样式单位使用不当（如px未转换为rpx）	1. 升级小程序基础库至2.20.0以上；2. 按需引入组件减小包体积；3. 统一使用rpx作为样式单位
uni-gallery图片无法预览	1. 图片数组格式错误，未包含url和thumbnail字段；2. 图片路径为相对路径或无效路径	1. 确保图片数组每个元素包含url和thumbnail属性；2. 使用绝对路径或Base64格式图片，避免相对路径

六、总结

本方案在萌宠案例中集成了uni-ui的多个核心组件，覆盖了从分类页到详情页的完整业务流程，核心优势如下：

1. **开发效率提升：**uni-ui组件封装了复杂的交互逻辑（如图片预览、标签切换），无需重复开发，减少代码量达30%以上。
2. **交互体验优化：**组件自带的过渡动画、状态反馈（如加载、空数据），让页面更具质感，提升用户体验。
3. **跨端兼容性好：**uni-ui组件经过多端适配测试，在小程序、App、H5端表现一致，避免针对性适配的繁琐工作。
4. **可维护性强：**组件化开发使代码结构清晰，后续修改（如调整搜索框样式、修改列表布局）只需操作对应组件，降低维护成本。

后续可根据业务需求引入更多uni-ui组件，如uni-swipe-action（滑动删除列表项）、uni-pagination（分页控件）等，进一步丰富萌宠案例的功能与交互体验。

（注：文档部分内容可能由 AI 生成）